

COSC 3750 — Systems Programming

Final Project | 200 Points

Overview

In this assignment you will design and implement a working Unix shell in Go. Your shell must support an interactive read-eval-print loop (REPL), a complete set of built-in commands, chained pipeline execution, output redirection, and a persistent command history. The project is deliberately modeled after the kind of shell implementation exercise common in C-based systems programming courses — the same POSIX-level concepts apply in Go, and you are expected to make those connections explicit in your code and documentation.

Chapter 6 of *Hands-On System Programming with Go* ("Building Pseudo-Terminals") provides the architectural foundation you should study before writing a single line of code. Pay close attention to the extensible command pattern, input management via selector, and the distinction between volatile and persistent command state. Your implementation must reflect this architecture — not replicate the book's code verbatim, but demonstrate that you understand and have applied its design principles to a more complete system.

This is an individual assignment. No group submissions will be accepted.

I/O Note

A note on I/O discipline: As with prior assignments, `fmt.Println`, `fmt.Print`, and `fmt.Printf` are disallowed throughout this project. All output must be written explicitly using the `io.Writer` provided to your command function, or to `os.Stderr` for error output. When you need formatted output, use `fmt.Fprintf(stdout, ...)` or `fmt.Fprintf(os.Stderr, ...)` passing the appropriate writer explicitly. This is not a stylistic preference — in a pipeline, your command's stdout writer is wired to the next stage's stdin reader by the pipeline executor. A command that bypasses that writer and prints directly to the terminal breaks the pipeline silently and will be penalized. The same rule applies to reading: do not read from `os.Stdin` directly. Use the `io.Reader` passed to your command function, except where this document explicitly carves out a terminal read (the `rm -r` confirmation prompt).

Learning Objectives

- Implement a functioning interactive shell using Go's `os` package and standard I/O interfaces
- Understand how the POSIX pipeline model maps to in-process goroutines and `io.Pipe()` in Go
- Implement chained pipelines by wiring command functions together via `io.Reader`/`io.Writer` pairs
- Implement output redirection using file descriptors, mapping to POSIX `open(2)` semantics
- Apply filesystem syscall semantics (`open`, `mkdir`, `rmdir`, `unlink`, `chdir`, `stat`) through Go's standard library
- Manage persistent state across sessions using file I/O
- Design a modular, package-based command dispatch architecture suitable for a real tool
- Demonstrate proper error propagation, stderr discipline, and exit-code handling throughout

Required Project Structure

Your submission must follow this exact directory and package layout. Every directory listed below corresponds to a distinct Go package with its own package declaration. Deviating from this structure without written approval will result in deductions on both the project structure line item and the architecture rubric category.

```
repository/  
  main.go           // Minimal entry point only – see below  
  README.md         // Required documentation – see README section  
  go.mod  
  
  internal/  
    shell/           // Shell struct, REPL loop, prompt, state management  
    dispatch/        // Command registration table and lookup logic  
    pipeline/         // Pipeline parsing and multi-stage execution  
    redirect/         // Output redirection parsing and file wiring  
    history/          // History load, save, and append operations  
  
  commands/  
    cat/              // Integrated from prior homework  
    ls/               // Integrated from prior homework  
    cd/  
    pwd/  
    echo/  
    touch/  
    mkdir/  
    rm/  
    rmdir/  
    grep/  
    history/  
    exit/  
    yourchoice/
```

main.go — Minimal Entry Point

`main.go` must be a minimal entry point. Its only responsibilities are instantiating the shell from `internal/shell`, calling the shell's `Run()` method, and passing the returned exit code to `os.Exit()`. It must contain no command logic, no input parsing, and no direct filesystem operations. A well-written `main.go` for this project should be in the range of 15–25 lines. If yours is significantly longer, that is a signal that logic which belongs in `internal/shell` has leaked into the entry point, and points will be deducted on the architecture rubric accordingly.

Why does this matter? In real systems software — shells, daemons, network servers — `main()` is conventionally a thin bootstrap. The substance lives in well-defined packages with clear interfaces. This discipline makes components testable in isolation and the program easier to reason about. You are being held to that standard here.

Internal Packages (`internal/`)

The `internal/` directory contains the core machinery of your shell. These packages handle state, dispatch, execution, and I/O plumbing. None of these packages should contain user-facing command logic — that belongs in `commands/`.

`internal/shell`

This is the heart of your program. The shell package defines a `Shell` struct that encapsulates all mutable shell state. Storing state in global variables is not permitted and will be penalized on the architecture rubric.

The `Shell` struct must track at minimum:

- `cwd string` — the current working directory, kept in sync with the process via `os.Chdir()`
- `history []string` — the in-memory command history for the current and prior sessions
- `lastExitCode int` — the exit code returned by the most recently executed command
- `dispatcher *dispatch.Dispatcher` — a reference to the command dispatch table

The shell package must expose a `Run()` method that drives the REPL loop: print the prompt, read a line of input, parse it, dispatch it, record it in history, and repeat until exit or EOF. The `Run()` method is what `main.go` calls — all REPL logic lives here, not in `main`.

`internal/dispatch`

The dispatch package implements the command registration table. Commands must not be selected with a bare if/else chain or switch statement in the REPL loop. Instead, commands are registered by name at startup and looked up by name at runtime.

All commands must share a consistent function signature. The required signature is:

```
type CommandFunc func(args []string, stdin io.Reader, stdout io.Writer) error
```

This signature is what makes pipelines possible: each stage receives its stdin as an `io.Reader` and writes output to an `io.Writer`, so the pipeline executor can wire stages together without any command knowing anything about its neighbors.

The `Dispatcher` must support: registering a command by name, looking up a command by name, and returning a clear error for unknown command names.

`internal/pipeline`

The pipeline package is responsible for parsing pipeline syntax and executing multi-stage pipelines. A pipeline is any command line containing one or more `|` characters.

The pipeline executor must:

- Split the input line on `|` to produce an ordered list of stage specifications
- For each stage, look up the command name in the dispatch table
- Create an anonymous pipe between each adjacent pair of stages

- Wire each stage's stdout to the next stage's stdin through its pipe
- Launch all stages as goroutines before waiting on any of them — all stages must run concurrently so that upstream stages can stream output to downstream stages without blocking
- Use a `sync.WaitGroup` or channel-based mechanism to wait for all goroutines to complete
- Collect and report any errors returned by each stage to stderr

POSIX connection: A real shell pipeline calls `pipe(2)` to create a file descriptor pair, then `fork(2)/exec(2)` for each stage, using `dup2(2)` to rewire the stage's stdin/stdout onto the pipe ends before `exec`. Your implementation achieves the same data flow entirely within a single process: `io.Pipe()` replaces `pipe(2)`, goroutines replace `fork(2)/exec(2)`, and passing the pipe's reader/writer as function arguments replaces `dup2(2)`. The logical model is identical; the mechanism is in-process. Your README and code comments should make this analogy explicit.

Pipeline execution must compose with redirection: a pipeline ending in a redirection operator (e.g., `cmd1 | cmd2 > file.txt`) must direct the final stage's stdout to the target file rather than to the terminal. The pipeline and redirect packages must cooperate to handle this case.

Common deadlock: if you run pipeline stages sequentially — calling each command function to completion before starting the next — an upstream stage will block trying to write to a full pipe buffer while you are waiting for it to return. All stages must be launched as goroutines before any are waited on.

internal/redirect

The redirect package handles output redirection. It is responsible for recognizing redirection tokens in the parsed command line, opening the target file with the correct flags, and returning an `io.Writer` that the pipeline executor uses as the final stage's stdout.

Your shell must support the following redirection operators:

- `> file` — redirect stdout to file, creating it if it does not exist and truncating it if it does. Wraps `os.OpenFile` with `os.O_WRONLY|os.O_CREATE|os.O_TRUNC` and mode `0644`. POSIX: `open(2)` with `O_WRONLY|O_CREAT|O_TRUNC`.
- `>> file` — redirect stdout to file in append mode, creating it if it does not exist. Wraps `os.OpenFile` with `os.O_WRONLY|os.O_CREATE|os.O_APPEND` and mode `0644`. POSIX: `open(2)` with `O_WRONLY|O_CREAT|O_APPEND`.

Redirection must not affect error output. Errors must always be written to `os.Stderr` regardless of any active redirection — this is correct Unix behavior and will be explicitly tested.

internal/history

The history package manages the shell's command history across sessions. It must provide the following exported functions:

- `Load(path string) ([]string, error)` — read history from a plain text file, one command per line, and return the entries as a string slice
- `Save(path string, entries []string) error` — write all history entries to a file, one per line, overwriting any existing content
- `Append(entries []string, cmd string) []string` — append a new command to the in-memory slice and return the updated slice

The history file must be stored at `~/.gosh_history`. History must be loaded at shell startup before the first prompt is displayed. History must be written to disk when the shell exits, whether via the `exit` command or via EOF (Ctrl+D). A missing history file at startup is not an error — the shell simply starts with an empty history.

Command Packages (`commands/`)

Each command lives in its own package under `commands/` and exports a single function conforming to the `CommandFunc` signature defined in `internal/dispatch`. This function is what gets registered in the dispatch table at startup.

Two rules apply universally to every command package:

- Commands must use only the `stdin` and `stdout` parameters they are given. Direct reads from `os.Stdin` or writes to `os.Stdout` are not permitted — doing so breaks pipeline composability.
- **Errors must be written to `os.Stderr`, never to the stdout writer parameter. Mixing error output into stdout will corrupt pipeline data and will be penalized.**

`commands/cat` — Concatenate Files

Integrate your `cat` implementation from the prior homework assignment. The integrated version must conform to the `CommandFunc` signature, receiving its output writer as a parameter rather than writing to `os.Stdout` directly. `cat` must function correctly as a pipeline source and as a passthrough stage.

- **Arguments:** zero or more file paths
- **With file arguments:** open and stream each file's contents to the stdout writer in argument order
- **With no arguments:** copy from the stdin reader to the stdout writer (passthrough mode — enables use as a middle pipeline stage)
- **Error handling:** if a file cannot be opened, print the error to stderr and continue to the next file without aborting

`commands/ls` — List Directory Contents

Integrate your `ls` implementation from the prior homework assignment, adapted to the `CommandFunc` signature. `ls` must function correctly as a pipeline source — its output must flow downstream when used in a pipeline.

- **Arguments:** optional directory path; defaults to the current working directory if omitted
- **Output:** one entry per line, written to the stdout writer
- All flags implemented in the prior homework must continue to work
- **Error handling:** if the target directory cannot be read, print a descriptive error to stderr

`commands/cd` — Change Directory

`cd` changes the shell's current working directory. It is one of the few commands that must mutate shell state, which creates a design challenge: the `CommandFunc` signature does not naturally provide access to the shell struct. You must decide how `cd` communicates the new directory back to the shell. Common approaches include registering `cd` as a closure that captures a pointer to the shell state, or defining a small state-access interface that the shell implements.

- **Arguments:** exactly one path argument; with no argument, change to the user's home directory (resolve with `os/user`)
- Calls `os.Chdir()` to update the actual process working directory — POSIX: `chdir(2)`
- The shell's `cwd` field and the displayed prompt must both reflect the new directory immediately after a successful `cd`
- If the target path does not exist or is not a directory, print a descriptive error to `stderr` and leave the working directory unchanged

commands/pwd — Print Working Directory

`pwd` prints the shell's current working directory. It exists to demonstrate that CWD is process-level state — the value printed must always match the path shown in the prompt, and both must reflect any prior `cd` invocations.

- **Arguments:** none
- Calls `os.Getwd()` and writes the result to the `stdout` writer followed by a newline
- Must function correctly as a pipeline source: `pwd | grep home`

commands/echo — Print Arguments

`echo` writes its arguments to `stdout` separated by spaces, followed by a newline. It is a required pipeline source — you will rely on it heavily when testing pipelines and redirection during development.

- **Arguments:** zero or more strings
- Join all arguments with a single space and write the result to the `stdout` writer
- With no arguments, print an empty line
- Must function correctly as the first stage of a pipeline: `echo hello world | grep hello`

commands/touch — Create or Update File

`touch` creates an empty file at the given path if it does not exist. If the file already exists, `touch` updates its access and modification timestamps to the current time without altering its contents. This command directly exercises POSIX file creation semantics.

- **Arguments:** one or more file paths
- **Non-existent file:** use `os.OpenFile` with `os.O_CREATE|os.O_WRONLY` and mode `0644` — POSIX: `open(2)` with `O_CREAT`
- **Existing file:** use `os.Chtimes` with the current time to update timestamps — POSIX: `utimensat(2)`
- Process all path arguments in order; a failure on one path must not prevent processing the remaining paths
- Print a descriptive error to `stderr` for any path that cannot be created or updated

commands/mkdir — Create Directory

`mkdir` creates a new directory at the specified path. It is a thin wrapper over the POSIX `mkdir(2)` syscall and demonstrates how Go's standard library maps directly to kernel interfaces.

- **Arguments:** one or more directory paths
- Uses `os.Mkdir` with mode `0755` — POSIX: `mkdir(2)`

- If the directory already exists, print an informative error to stderr but do not treat this as a fatal failure — continue processing remaining arguments
- If the parent path does not exist, print an error. This implementation does not need to support `-p` recursive parent creation.
- Process all arguments in order

commands/rm — Remove Files and Directories

`rm` removes files and, with the `-r` flag, entire directory trees. This command requires the most careful error handling in the project — a defect here can cause unrecoverable data loss, and that is true whether the language is C or Go.

- **Without `-r`:** removes a single file. Uses `os.Remove` — POSIX: `unlink(2)`
- **With `-r`:** recursively removes a directory and all of its contents. Uses `os.RemoveAll` — equivalent to C's `nftw()` with `unlink/rmdir`
- **Confirmation required for `-r`:** before any recursive removal, print the prompt `Remove directory '<path>' and all contents? [y/N]:` and read from the terminal. Proceed only if the user enters `y` or `Y`. Any other input, including empty, must abort the operation.
- If the target path does not exist, print a descriptive error to stderr
- `rm` must never silently succeed or silently fail — every operation must produce either visible confirmation or a clear error

Note: the `-r` confirmation prompt reads directly from the terminal, not from the command's stdin parameter. This is an intentional exception to the "no direct terminal reads" rule and mirrors how real `rm -i` behaves.

commands/rmdir — Remove Empty Directory

`rmdir` removes a directory only if it is empty. It must refuse to remove a non-empty directory — this is the key behavioral distinction from `rm -r`, and it reflects the constraint enforced at the kernel level by the POSIX `rmdir(2)` syscall.

- **Arguments:** one or more directory paths
- Uses `os.Remove`, which calls `rmdir(2)` on directories — POSIX: `rmdir(2)`
- If the directory is non-empty the syscall fails; catch this error and print `rmdir: '<path>': Directory not empty` to stderr
- If the path does not exist or is not a directory, print a descriptive error to stderr
- Process all arguments in order

commands/grep — Search Lines by Pattern

`grep` reads lines from its stdin and writes only the matching lines to its stdout. It is the most important pipeline stage in the project — virtually every non-trivial pipeline test will involve `grep`. It must function correctly whether it receives input from a file (via `cat`), from another command, or directly from terminal input.

- **Arguments:** one required pattern; no flag support is required
- Treat the pattern as a literal substring. A line matches if it contains the pattern anywhere (case-sensitive).

- **grep** must function correctly at any position in a pipeline: as a middle stage, as a final stage, and with output redirection applied to it

commands/history — Display Command History

The **history** command displays the shell's accumulated command history with numbered entries. It reads from the in-memory history slice maintained by the shell — not directly from the history file.

- **Arguments:** none
- Print each entry in the format **N command** where N is the 1-based index right-aligned in a field of width 4
- Entries from previous sessions appear first (in the order they were loaded), followed by commands from the current session
- Must function correctly as a pipeline source: **history | grep cd**

Design note: because **history** needs access to the shell's history slice, it faces the same design challenge as **cd**. The recommended approach is to register **history** as a closure that captures a pointer to the shell state when commands are registered at startup.

commands/exit — Exit the Shell

exit terminates the shell process. Before exiting it must flush the in-memory history to disk so that the current session's commands are available in future sessions. This is the only command that terminates the REPL loop via the normal code path — EOF (Ctrl+D) is the other path and must also trigger a history flush, handled in **internal/shell**.

- **Arguments:** optional integer exit code; defaults to 0 if omitted
- Must call the history package's **Save** function with the current history slice before exiting
- Must call **os.Exit** with the specified exit code after saving history
- If the exit code argument is present but is not a valid integer, print an error to stderr and exit with code 1

Optional Commands

You must select one of the following to implement.

commands/wc — Word Count

wc reads from its stdin or a file argument and prints counts of lines, words, and bytes. It is a natural pipeline stage and a classic Unix utility whose simplicity belies how frequently it appears in real shell usage.

- **Arguments:** optional file path; with no argument reads from the stdin reader
- **Flags:** **-l** print line count only; **-w** print word count only; **-c** print byte count only; with no flag print all three in the format **lines words bytes**
- Read input using a **bufio.Scanner** on the stdin reader or the opened file
- Output must be written to the stdout writer so it functions correctly as a pipeline stage: **cat file.txt | wc -l**
- With no flag, format output as **%8d %8d %8d** for lines, words, and bytes respectively, matching standard **wc** output alignment
- **Error handling:** if a file argument is given and cannot be opened, print a descriptive error to stderr

commands/head — Output First N Lines

head reads from its stdin and writes only the first N lines to stdout. It is most useful as an early pipeline stage to limit the volume of data flowing downstream.

- **Arguments:** optional file path; with no argument reads from the stdin reader
- **Flags:** **-n N** specifies the number of lines to output; defaults to 10 if omitted
- Read input line by line using a **bufio.Scanner**; stop reading and return after N lines have been written
- Must function correctly at any pipeline position: `cat large_file.txt | head -n 5 | grep pattern`
- If the input has fewer than N lines, write all lines and exit cleanly — this is not an error
- **Error handling:** if **-n** is given a non-integer argument, print a descriptive error to stderr and return a non-zero error

commands/tail — Output Last N Lines

tail reads from its stdin and writes only the last N lines to stdout. Unlike **head**, it must consume the entire input before it can produce output, which makes its buffering behavior an interesting implementation challenge.

- **Arguments:** optional file path; with no argument reads from the stdin reader
- **Flags:** **-n N** specifies the number of lines to output; defaults to 10 if omitted
- Read all input lines into a buffer, then write only the last N lines to the stdout writer
- Must function correctly as a pipeline stage: `history | tail -n 20`
- If the input has fewer than N lines, write all lines and exit cleanly
- **Error handling:** if **-n** is given a non-integer argument, print a descriptive error to stderr and return a non-zero error

commands/sort — Sort Input Lines

sort reads all lines from its stdin, sorts them, and writes the sorted result to stdout. Because it must consume all input before producing output, it naturally acts as a pipeline barrier — nothing downstream receives data until all upstream data has been read.

- **Arguments:** none; reads entirely from the stdin reader
- **Flags:** **-r** reverse the sort order; **-n** sort numerically rather than lexicographically (lines are parsed as integers for comparison)
- Read all lines into memory using a **bufio.Scanner**, sort the slice, then write each line to the stdout writer
- Must function correctly as a middle pipeline stage: `cat file.txt | sort | uniq`
- If **-n** is specified and a line cannot be parsed as an integer, treat it as zero for sorting purposes and continue — do not abort
- **Error handling:** no flags other than **-r** and **-n** are required; unknown flags should print a descriptive error to stderr

commands/uniq — Deduplicate Adjacent Lines

uniq reads from its stdin and writes each line to stdout, suppressing consecutive duplicate lines. It does not sort its input — it only collapses adjacent identical lines, which is why it is almost always preceded by **sort** in

real usage.

- **Arguments:** none; reads entirely from the stdin reader
- **Flags:** none required
- Read input line by line using a `bufio.Scanner`; track the previous line and skip writing the current line if it is identical to the previous one
- Must function correctly as a pipeline stage: `cat file.txt | sort | uniq`
- An empty input must produce empty output cleanly with no error
- Must function correctly as a pipeline source for further stages: `sort file.txt | uniq | wc -l`

commands/stat — Display File Metadata

`stat` prints detailed metadata about a file or directory by calling `os.Stat()` and formatting the returned `fs.FileInfo`. It has no pipeline role — it writes directly to stdout and is not designed to be piped — but it is one of the most direct demonstrations in the project of how Go's standard library surfaces POSIX `stat(2)` data.

- **Arguments:** one or more file or directory paths
- For each path, call `os.Lstat()` (which does not follow symlinks) and print the following fields, each on its own line:
 - **File:** the path as given
 - **Size:** size in bytes
 - **Mode:** the permission bits in octal and symbolic form (e.g., `0644 (-rw-r--r--)`)
 - **Modified:** the modification timestamp formatted as `2006-01-02 15:04:05`
 - **Type:** one of `regular file`, `directory`, `symlink`, or `other`
- Uses `os.Lstat()` — POSIX: `lstat(2)`
- Process all path arguments in order; a failure on one path must not prevent processing the rest
- **Error handling:** if a path does not exist or cannot be accessed, print a descriptive error to stderr and continue

Shell Prompt and REPL Behavior

Prompt Format

Your shell must display the following prompt format before each command:

```
username:cwd$
```

- **username** — the current user's login name, resolved via the `os/user` package at startup
- **cwd** — the current working directory. The home directory prefix must be displayed as `~` (e.g., `~/src` instead of `/home/jsmith/src`)
- The prompt must update immediately after every successful `cd` — it must never display a stale directory
- The prompt must be written to stderr, not stdout, so it does not appear in redirected output or pipeline data

REPL Loop Requirements

- Run continuously until the user types `exit` or sends EOF (Ctrl+D)
- Each input line must be parsed to identify: redirection tokens, pipeline stage boundaries (`|`), and per-stage argument lists
- Empty lines must be silently ignored — no error output, no history entry
- Unknown command names must print `'<cmd>': command not found` to stderr and continue the loop
- The exit code of the last executed command must be tracked in the shell struct after every dispatch
- Every successfully parsed and dispatched command — including ones that return errors — must be appended to the in-memory history

Grading

Features: 165 points | Rubric: 20 points | Optional command: 15 points | Total: 200

Feature Checklist (165 points)

Component	Pts	Criteria
REPL loop & prompt format	10	Correct <code>username:cwd\$</code> format; updates on <code>cd</code> ; empty lines ignored; unknown commands reported to stderr
<code>commands/cat</code> (integration)	10	Prior HW works under <code>CommandFunc</code> signature; correct as pipeline source and passthrough stage
<code>commands/ls</code> (integration)	10	Prior HW works under <code>CommandFunc</code> signature; output pipeable downstream; prior flags functional
<code>commands/cd + pwd + echo</code>	15	CWD updates in both process state and prompt; <code>cd</code> with no args goes to home; <code>echo</code> pipelines as source
<code>commands/touch + mkdir</code>	15	Correct creation and timestamp semantics; edge cases handled (existing file/dir); multiple args processed
<code>commands/rm + rmdir</code>	15	File removal correct; <code>-r</code> confirmation prompt required; <code>rmdir</code> rejects non-empty dirs with correct error
<code>commands/grep</code>	15	Literal substring match correct; works at any pipeline position
<code>commands/history</code>	15	Numbered display; loads from <code>~/.gosh_history</code> at startup; saves on exit; functions as pipeline source
Pipeline execution (chained)	20	Arbitrary-length chains; stdout→stdin wiring; all stages concurrent; exit codes propagated
Output redirection (<code>></code> and <code>>></code>)	20	Truncate and append modes correct; composes with pipelines; errors still go to stderr
<code>commands/exit</code> + history flush	10	Exit code argument accepted; history written before process terminates; EOF path also flushes
Project structure	10	All required packages present; <code>main.go</code> is minimal entry point; no logic leaked into main

Optional Command (15 points)

Implement one command from the optional list: `wc`, `head`, `tail`, `sort`, `uniq`, or `stat`. Declare your choice in your README. Only one optional command will be graded — if you implement more than one, indicate which you want graded.

Component	Pts	Criteria
Optional command (student's choice)	15	Correct behavior per spec; conforms to <code>CommandFunc</code> signature; error handling present; pipeline composability where applicable

Code Quality Rubric (20 points)

Component	Pts	Criteria
Error handling & stderr discipline	10	All errors checked; errors written to stderr not stdout; no panics on malformed input
Architecture & dispatch pattern	10	Dispatch table used; <code>CommandFunc</code> signature consistent throughout; no global state; pipeline decoupled from commands

Total: 200 points

README Requirements

Include this in your project, an extended version will also be submitted as a final report

Your `README.md` must contain the following five sections. A README that is missing sections or contains only placeholder text will receive zero points for the README rubric line regardless of code quality.

- Design Overview** — Describe your dispatch architecture, how pipeline execution works, and how shell state is managed. Write 2–4 paragraphs of substantive explanation, not a list of filenames.
- POSIX Syscall Map** — A table with three columns: Command | POSIX Syscall(s) | Go Standard Library Call. Every command that touches the filesystem or process state must have an entry.
- Known Limitations** — Honest documentation of any features that are incomplete, broken, or not implemented. An empty section here when problems exist will be treated as academic dishonesty.
- Build & Run** — Exact commands to build and launch the shell from a clean checkout. Must work without modification.
- Sample Session** — A terminal transcript showing: at least one chained pipeline (3+ stages), a redirection using `>>`, a `cd` followed by `pwd`, `history` output, and `rm -r` with the confirmation prompt.

Submission

- Submit all code to the GitHub repository
- Submit the repository URL and readme to WyoCourses

Due Date

End of Week 14 — see WyoCourses for the exact timestamp. Late submissions are penalized 10 points per calendar day.

Academic Integrity

You may reference Chapter 6 of the Go textbook and Go standard library documentation freely. You may discuss design approaches with classmates at a conceptual level. All code must be your own. Do not copy implementations from GitHub, online resources, or other students. Submissions will be reviewed for originality. Any suspected violation will be referred to the University of Wyoming Academic Dishonesty Policy.

Do not have AI write your code. This is a capstone course, it is the culmination of your experience at UW. You want to take away and learn from this course. You want to go to a future employer and talk about what **you** built. Adhere to the AI guidelines in the syllabus. You can use generative AI to help you learn, not just have it do it for you.

Implementation Guidance

Recommended build order — each step produces something testable before moving on:

1. Set up all required packages with stub functions so the project compiles cleanly from the start — import errors should never block progress on individual commands
2. Implement `internal/shell` and `internal/dispatch` — get the REPL loop printing the prompt and dispatching to stubs before writing any real command logic
3. Implement `commands/echo` and `commands/pwd` — they are the simplest commands and immediately useful for verifying dispatch is working
4. Implement `commands/cd` alongside `pwd` — test them together since `cd` must update the prompt and `pwd` must reflect the change
5. Implement `commands/touch` and `commands/exit` — touch exercises file creation early; exit ensures history is wired correctly from the start
6. Implement `internal/history` and connect it to the shell — verify history survives a restart before moving on
7. Integrate `cat` and `ls` from prior homework under the `CommandFunc` signature
8. Implement `commands/mkdir`, `commands/rmdir`, and `commands/rm` — test edge cases (existing dirs, non-empty dirs, confirmation prompt) before moving to text processing
9. Implement `commands/grep` — verify literal and `-E` modes work correctly in isolation
10. Implement your optional command
11. Implement `internal/pipeline` starting with single-stage execution (no `|` yet) to validate the `CommandFunc` wiring, then extend to arbitrary chained stages
12. Implement `internal/redirect` (`>` and `>>`), then test pipeline + redirection composition together
13. Write `README.md` and do final polish — confirm `go build ./...` is clean and test your submission archive from scratch

Common Bugs to Avoid

- Not waiting on all pipeline goroutines — use a `sync.WaitGroup` and ensure every goroutine is accounted for, otherwise the program can return before stages finish writing their output
- Writing error messages to the stdout writer instead of `os.Stderr` — this corrupts pipeline output and is the most common source of mysterious test failures

- Tracking CWD as a string without calling `os.Chdir()` — your stored path and the actual process CWD will diverge, causing all relative path operations to fail silently
- Not closing the write end of an `io.Pipe()` after the upstream stage's goroutine finishes — the downstream stage's `bufio.Scanner` will block indefinitely waiting for input that will never arrive
- Running pipeline stages sequentially rather than as concurrent goroutines — this deadlocks as soon as the upstream stage's output fills the pipe's internal buffer
- Forgetting to flush history on EOF — the `exit` command path and the Ctrl+D path are separate code paths and both must save history

COSC3750 Final Project: Suggested Timeline

This timeline is a recommendation, not a requirement. It is designed to distribute work evenly across the three weeks and to sequence components so that each week builds on the last. Students who fall behind in Week 1 will find Week 3 significantly harder — the pipeline and redirection work assumes all commands are already functioning correctly.

Week 1 — Shell Foundation and Core Commands

The goal this week is a working shell with a complete REPL loop and the simpler commands in place. By the end of Week 1 you should be able to start your shell, navigate the filesystem, and see history persisting across sessions.

- Set up project structure and `go.mod` with all required packages stubbed out
- Implement `internal/shell` — Shell struct, REPL loop, and prompt display
- Implement `internal/dispatch` — command registration table and `CommandFunc` signature
- Implement `internal/history` — load, save, and append; wire into shell startup and exit
- Integrate `cat` and `ls` from prior homework assignments under the `CommandFunc` signature
- Implement `commands/cd` — including prompt update and home directory fallback
- Implement `commands/pwd`
- Implement `commands/echo`
- Implement `commands/touch`
- Implement `commands/exit` — with history flush on both `exit` and EOF (Ctrl+D)

Week 1 checkpoint: your shell should start, display the correct prompt, update the prompt on `cd`, print the working directory with `pwd`, echo arguments, create files with `touch`, maintain a history that survives restart, and exit cleanly.

Week 2 — Filesystem Commands and Text Processing

With the shell foundation in place, this week adds the remaining individual commands. Each command this week is self-contained and testable in isolation before pipelines exist.

- Implement `commands/mkdir`
- Implement `commands/rmdir`
- Implement `commands/rm` — including `-r` recursive removal with confirmation prompt
- Implement `commands/grep` — literal match

- Implement your chosen optional command (`wc`, `head`, `tail`, `sort`, `uniq`, or `stat`)

Week 2 checkpoint: all required commands work correctly when invoked individually. You can create and remove files and directories, search text, and your optional command produces correct output. No pipeline support is required yet — test each command by running it directly.

Week 3 — Pipelines, Redirection, and Final Polish

This week adds the plumbing that connects everything. Pipeline and redirection work is the most technically demanding part of the project — do not leave it to the last day.

- Implement `internal/pipeline` — single-stage execution first, then extend to arbitrary chained stages
- Implement `internal/redirect` — `>` truncate and `>>` append modes; compose with pipeline final stage
- Test pipeline + redirection composition thoroughly: `cat file.txt | grep pattern | wc -l >> results.txt`
- Write `README.md` — all five required sections including the POSIX syscall map and sample session
- Final polish: error messages, edge cases, confirm build is clean with `go build ./...`
- Package and test your submission archive before the deadline

Week 3 checkpoint: arbitrary-length pipelines work correctly, output redirection composes with pipelines, errors still go to stderr under redirection, and your README is complete. Extract your archive to a clean directory and confirm it builds and runs from scratch before submitting.

General Advice

- **Stub early.** Get all packages compiling with placeholder functions in Week 1 so you are never blocked by import errors while working on individual commands.
- **Test in isolation first.** Every command should be verified working on its own before you wire it into pipeline tests.
- **Pipelines amplify bugs.** A command that writes errors to stdout instead of stderr will appear to work fine in isolation but will corrupt pipeline output silently. Fix stderr discipline before Week 3.
- **The README takes longer than you expect.** The POSIX syscall map and sample session in particular require time to do properly. Do not leave the README for the final night.